Names: Savit Tumuluri, Samii Shabuse, Han Truong

DSCI351 Assignment 2

Format is question as first bullet point, answers in rest

Step 1:

- Construct a rating matrix with the dataset provided, each row refers to user id, each column refers to movie id. Which movie has the highest number of known ratings? Provide movie id, movie title and known rating number.
- Movie ID: 2858
- Movie Title: American Beauty (1999)
- Number of Ratings: 3428

Step 2:

- Split the rating matrix into X and Y. Where Y is the column of the movie with the highest number of known ratings in the previous step. X is the resting of the columns in rating matrix. After the split, for both X and Y, only keep the rows where have known rating in Y. What is the dimension of X now?
- X dimension: (3428, 3705)

Step 3:

- Let's perform further cleaning with Y. The known ratings range from 1 to 5. For this model-based movie recommender, we would like to set a threshold, if the rating is higher than (>) the threshold, then we consider the movie can be recommended, otherwise (<=), not recommended. Suppose the threshold is 3, change the value in Y based on such preference. What is the portion of movie to be recommended in Y? What is the sparsity of X? Then, impute 0 for unspecified ratings in X for modeling.
- Portion of users that recommended the movie in Y: 0.8322637106184364
- Sparsity of X: 0.9437629618431682

Step 4:

- Split the X and Y in train set (80%, X_train, Y_train) and test set (20%, X_test, Y_test), make sure your data split can be replicated. What is the dimension of X_test now?
- X_test dimension: (686, 3705)
- A fixed random_state (42) is used to ensure the train-test split is reproducible.

Step 5:

- Now let us train a Naïve Bayes model. Look into the document from scikit-learn regarding MultinomialNB and BernoulliNB, and observe your dataset, which Naïve

Bayes method should be applied? Provide your explanation. With selected Naïve Bayes methods, with hyper-parameter setting as alpha=1 and fit_prior=True, train with your train set (X_train, Y_train).

- The bayes method that should be applied is the Multinomial Naives Bayes model. This is because the feature matrix X contains nonnegative numerical values, which are ratings from 0 to 5 after imputing missing values as 0, instead of using binary values. The model Bernoulli is better used for binary features, while MultinomialNB is better for count-based or nonnegative data. There for this dataset Multinomial is better to be used.

Step 6a:

- With trained Naïve Bayes model, make prediction with your test set (.predict(X_test)). What is the percentage of instances in X_test are correctly classified, when compare preditions to Y_test?
- Using the trained MultinomialNB model, the percentage of correctly classified instances in X_test is 71.57%.

Step 6b:

- Now check the .predict_proba(X_test) output with trained Naïve Bayes model, each row has two probabilities (probability of the instance belong to class 0, and probability of the instance belong to class 1), the sum of them should be equal to 1 in each row. Examine and compare the output to the predictions, what is threshold in 2nd probability result applied for classification prediction? If set the threshold as 0.6 to make the predicition, what is the percentage of instances in X_test are correctly classified now? With 2nd probability higher than threshold, predict as class 1, otherwise, predict as class 0.
- The default threshold in the 2nd prob results are 0.5 of the classification. This was then verified after manually testing the prediction against 0.5 which produced identical results.
- When then threhsold was increased to 0.6, the percentage of correctly classified instances became 70.70%, which mean increasing the threshold makes the model more conserative in predicting class 1, which reduced overall accuracy.

Step 7:

- Now use the splitted data from step 4, first apply sklearn.decomposition.TruncatedSVD(n_components = 3000) on train data (.fit_transform(X_train)) and test data (.transform(X_test)). Then use GaussianNB to train transformed train data (X_train_transformed, Y_train), and make predictions on transformed test data (.predict(X_test_transformed)). What is the percentage of instances in test data are correctly classified now? Provide discussion regarding the performance with trained Naïve Bayes models (including the whole procedure from raw data)
- The percentage of instances in the test data that are now correctly classified are 84.40%.
- The dataset was first transformed into a user–movie rating matrix, which was highly sparse (about 94% missing values). The task was converted into a binary classification problem based on whether a user would recommend the most-rated movie. A

Multinomial Naïve Bayes model achieved an accuracy of approximately 71.57%, which slightly decreased to 70.70% when the classification threshold was increased from 0.5 to 0.6.

To improve performance, TruncatedSVD was applied to reduce dimensionality and capture latent features. A Gaussian Naïve Bayes model trained on the transformed data achieved a significantly higher accuracy of approximately 84.40%. This improvement is due to reduced sparsity, better feature representation, and the suitability of GaussianNB for continuous-valued features.

Overall, dimensionality reduction combined with an appropriate model significantly improves performance compared to using the original sparse data. This demonstrates the importance of feature transformation and model selection in recommender systems, especially when dealing with sparse, high-dimensional data.